

S11BLH21- Programming in python



SCHOOL OF COMPUTING

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

UNIT-III – S11BLH21- Programming in python

FILES AND EXCEPTIONS HANDLING, MODULES, PACKAGES

File Operations –Iterators - Exception handling - Regular Expressions- Creating Modules- Import Statement Introduction to PIP-Installing Packages via PIP-Using Python Packages.

FILE OPERATIONS

An object that stores data, settings or programming commands in a computer system is called as a file. There are three major file operations:

- Opening a file
- Performing file operations using Read or Write
- Closing the file

File Open

Method:open()

Purpose: To open a file

Syntax:

```
File_object=open(filename,Access_mode,buffering)
```

Attributes:

- i. Filename – Name of the file

S11BLH21- Programming in python

- ii. Access_mode- Mode of Access (Read, Write, Append)
- iii. Buffering – 0 (no buffer), 1 (buffer)

Example:

```
f= open('abc.txt')      (or)  
f=open(,D:/Mypython/abc.txt')
```

File Access Modes

File Mode	Description
r	Read mode
w	Write mode
x	Create and open a file
a	Appending at end of file
t	Text mode
b	Binary mode
+	Update mode

Example:

```
f= open('abc.txt', r)
```

The above statement opens the file 'abc.txt' in read mode.

Example for File Access modes and Properties

```
fo=open('aa.txt','w')
print('Filename: ', fo.name)
print('Filemode: ', fo.mode)
print('File closed: ', fo.closed)
fo.close()
print('Fileclosed: ', fo.closed)

'''
output:
Filename:  aa.txt
Filemode:  w
File closed:  False
Fileclosed:  True
'''
```

The above code is a sample snippet for understanding the file modes and file properties.

File Reading and Writing

File write:

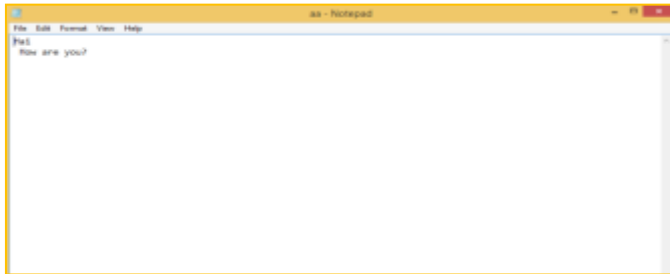
write() method is used to write the contents to a file. The following code is for writing the contents to the file aa.txt.

```
fo=open('aa.txt','w')

fo.write('hai \n how are you?')

fo.close()
```

Output:



In the above example, the contents of the file can be viewed by opening the file 'aa.txt'.

Reading a file:

read() method is used to read the contents from a file. The following code is for reading the first 10 bytes of the file 'aa.txt'.

```
fo=open('aa.txt','r')
print(fo.read())
#reading 10 bytes
fo.read(10)
fo.close()
'''
output:
Hai
How are you?
'''
```

File Positions

To know about the file offset positions in Python, the following methods are used:

- seek()
- tell()

seek():

Syntax: seek(offset, from)

Description: Sets the file's current position at the offset. The offset values are as follows:

0 : reference (beginning of file(default))

1 : current (current file position)

2 : end (end of file)

tell() :

Description: Prints the current position of file pointer.

2.1.3.1. File Offset

'h'	'a'	'i'		,		'h'	'o'	'w'		'a'	'r'	'e'		'y'	'o'	'u'	'?'
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

S11BLH21- Programming in python

```
fo=open('aa.txt','r')
print('current position',fo.tell())
print(fo.read(10))
print('current position',fo.tell())
fo.seek(2,0) # to skip first 2
print(fo.read())
...
output
current position 0
Hai , How
current position 10
i , How are you?
...
```

In the above code, initially the position of the file pointer is at 0. After reading the contents, the position of the file pointer is moved to 10 (from 0 to 9). Now up on giving the command seek(2,0), the file will be read from the beginning after skipping the first 2 positions.

Detailed Example:

```
f=open('aa.txt','r')
pos=f.tell()
print(pos)
#output : 0
line=f.readline()
print(line)
#output: prints first line Hai , How are you?
pos=f.tell()
print(pos)

#20
line=f.readline()
print(line)
#
f.seek(0,0)
pos=f.tell()
print(pos)
line=f.readline()

print(line)
pos=f.tell()
print(pos)
line=f.readline()
print(line)

... output
0
Hai , How are you?

20
Welcome to Sathyabama

0
Welcome to Sathyabama

43
School of Computing
...
```

S11BLH21- Programming in python

The contents of the file aa.txt is now:

```
Hai , How are you?  
Welcome to Sathyabama  
School of Computing  
Department of Computer Science & Engineering
```

Reading a file Line by line

In order to read a file till the End of File(EoF), while loop is used.

```
f=open('f8.txt','r')  
line=f.readline()  
while line!='':  
    print(line)  
    line=f.readline()  
f.close()  
'''output  
kdskf  
  
dsafldk  
  
kdafsljff'''
```

Modifying a file

```
f=open('aa.txt','a')  
f.write('aa bb cc dd')  
f.close()  
f=open('aa.txt','r')  
print(f.read())  
#prints the entire file  
  
#go to 5th position using seek(5)  
f.seek(5,0)  
print('from 5th poan',f.read())  
f.seek(30)  
line=f.readline()  
#prints poan of line from 30th poan  
print('line at 30', line)  
f.seek(0)  
#print(f.read())#prints full file  
print('current poan before reading',f.tell())  
f.close()  
'''  
output  
Hai , How are you?  
Welcome to Sathyabama  
School of Computing  
Department of Computer Science & Engineeringaa bb cc ddaa bb cc dd  
from 5th poan How are you?  
Welcome to Sathyabama  
School of Computing  
Department of Computer Science & Engineeringaa bb cc ddaa bb cc dd  
line at 30 Sathyabama  
  
current poan before reading 0  
...'''
```

S11BLH21- Programming in python

Alternate way for opening and closing a file:

Syntax:

with open('filename') as file object:

- No need to close the file

```
with open('aa.txt') as f:
    for line in f:
        print(line)

'''output
Hai , How are you?

Welcome to Sathyabama

School of Computing

Department of Computer Science & Engineeringaa bb cc ddaa bb cc dd'''
```

read() &readline()

- read() – read entire file content from current position
- readline() – read the particular line of file pointer

ITERATORS

Iterator in Python is a type which could be implemented in for loops. An iterator is an object that returns data one at a time.

For example if we have a list `A=[1,2,3]` , then iterator is used to return the items in the list one at a time.

S11BLH21- Programming in python

There are two special Methods:

- `__iter__()` : returns iterator from list
- `__next__()`: returns next element in the list

Iterable objects in Python are:

- List
- Tuple
- String

Example Iterator:

```
mylist=[4,7,0,3]
myiter=iter(mylist)
print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
print(next(myiter))
'''
output
4
7
0
3
Error
'''
```

In the above code the list items of mylist object are retrieved one by one using 'next()' method. When the list reaches its end and if next() method is used , it shows error in the output.

Example for `__next__()`

Alternate way for retrieving the items is to use for loop and retrieve the item using `__next__()` inside the for loop. To find the length of the list '`len()`' method is used.

```
list=[3,4,5,6]
iterobj=iter(list)
print()
for i in range(0,len(list)):
    print(iterobj.__next__())
'''
3
4
5
6
'''
```

Building User defined iterators

We can also build our own iterators. The following code is for implementing user defined iterators for finding powers of two.

S11BLH21- Programming in python

```
class pow2:
    """To implement an iterator of powers of two"""
    def __init__(self, max=0):
        self.max=max
    def __iter__(self):
        self.n=0
        return self
    def __next__(self):
        try:
            if self.n==self.max:
                res=2**self.n
                self.n+=1
                return res
            raise StopIteration
        except StopIteration:
            quit(0)

a=pow2(4)
i=iter(a)
print(next(i))
while True:
    print(next(i))
'''Output'''
1
2
4
8
16
'''
```

Python Infinite Iterators:

There are two Arguments in infinite iterators:

- Callable Object: A built in function
- Sentinels: The terminating value

The following is an example for infinite iterator. `next(inf)` will always return 0, since the sentinel 1 is not reached.

```
>>> int()
0
>>> inf=iter(int,1)
>>> next(inf)
0
>>> next(inf)
0
```

Similarly, the following code uses while loop to print the odd numbers starting from 1 to infinite number of times. The execution is manually terminated by providing keyboard interrupt(Ctrl+c).

S11BLH21- Programming in python

```
class infin:
    def __iter__(self):
        self.num=1
        return self
    def __next__(self):
        num=self.num
        self.num+=2
        return num
a=iter(infin())
while True:
    print(next(a))
'''output
1
3
5
7
9
11
13
15
17
19.....'''
```

Python Generators

Generator functions are alternates for iterators that contain one or more **yield()** statements. Methods like `__iter__()`, `__next__()` are implemented and are iterated using `next()` automatically. Local variables are remembered between successive calls. When function terminates, `StopIterator` exception is raised automatically.

Example

In the following code, `n` value is initiated to 1 in the first step. In the second step `n` is incremented by two and the value yielded now is 3. In the last step `n` is incremented by 1 and now the value is 4.

```
def my_gen():
    n=1
    print('first')
    yield n
    n+=2
    print('second')
    yield n
    n+=1
    print('last')
    yield n
for i in my_gen():
    print(i)

'''output
first
1
second
3
last
4'''
```

The following is an example for reversing a String using python Generator. Here the string 'hello' is passed to the function 'rev()'. Using for loop, the string is yielded from the last character(`len-1`) to `-1`(`0th` position minus 1) as per the syntax.

S11BLH21- Programming in python

```
def rev(mystr):
    len1=len(mystr)
    for i in range(len1-1,-1,-1)
        yield mystr[i]

for c in rev('hello'):
    print(c)

'''output
o
l
l
e
h'''
```

Advantages of Generators

- Easy to implement
- Memory efficient
- Represents infinite stream
- Generators can be pipelined

EXCEPTION HANDLING

Exception is an event that occurs during execution of a Python program disrupting the normal flow of execution. Exceptions are handled using try and except blocks in Python. There are built in exception classes for handling common exceptions. BaseException is the parent class for all built in Exception classes. Fig 2.1 represents the Standard Exception class hierarchy.

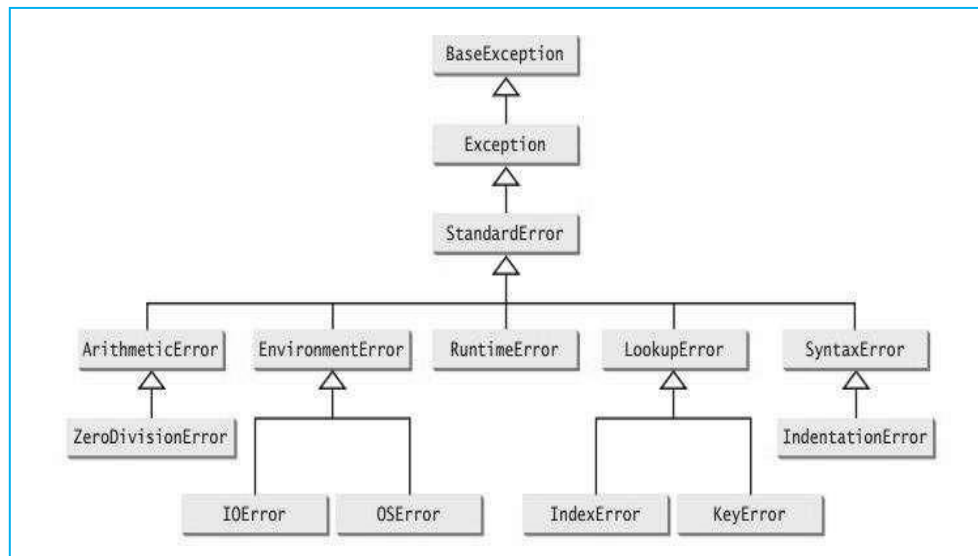


Fig 2.1 Standard Exception class hierarchy

Exception Handling Syntax and Examples

While handling exception, keep the suspicious code in try block and following the try block, include except: statement

```
try:
    suspicious block
except Exception1:
    #statement1
except Exception2:
    #statement2
.....
else:
    no exception
```

S11BLH21- Programming in python

The following code raises exception when a run time error occurs upon writing the file 'aa.txt'. In case of normal program flow, the else clause will be invoked and the statements in else block will be executed.

```
try:
    fo=open('aa.txt','w')
    fo.write('Exception for exception')
except IOError:
    print('cant write')
else:
    print('written successfully')

#output:
''' written successfully'''
#content has been written to file aa.txt
```

IOError exception is also invoked when we intend to write a file when it is opened in 'read' mode. The following code depicts this case.

```
try:
    fo=open('aa.txt','r')
    fo.write('Exception handling example')
except IOError:
    print('cant write in read mode')
else:
    print('written successfully')

#output:
''' cant write in read mode'''
```

Except Clause without specifying any exception

In python, we can also have except clause with no specific exception. In this case any type of exception can be handled. The following is the syntax for except statement with no specific exception type.

S11BLH21- Programming in python

Syntax:

```
try:
    #Error code
except:
    #Execute block with Any exception
else:
    #No exception
```

Example:

In the following code, except clause is alone given, without mentioning the type of exception. In the sample runs when the value of 'b' is given as 0, exception is caught and 'divide by zero error' is printed. Whereas, in case of normal run, the result obtained after dividing two numbers, is displayed as the output.

```
a,b=eval(input('Enter two nos.'))
try:
    c=a/b
except:
    print('divide by zero error')
else:
    print('Normal execution & the value is',c)

'''Sample outputs:
Run1:

Enter two nos.2,0
divide by zero error

Run 2:

Enter two nos.3,6
Normal execution & the value is 0.5
'''
```

Except Clause with Multiple exceptions:

There is another way of specifying multiple exceptions in the single except clause. When multiple exceptions are thrown, the first exception which is being caught will alone be handled. The syntax is given as follows.

Syntax:

```
try:
    #Error code
except (Exception 1, Exception2,...):
    #Execute block with Any exception
else:
    #No exception
```

Example:

```
a=input('Enter the value of a')
b =input('Enter the value of b')
try:
    c=a/b
except (TypeError, ZeroDivisionError):
    if TypeError:
        print('Type error')
    elif ZeroDivisionError:
        print('Divide by zero error')

else:
    print('Normal execution & the value is',c)

'''
Sample output:
Enter the value of a : 6
Enter the value of b : a
Type error
'''
```

S11BLH21- Programming in python

Optional finally clause

Like other object oriented programming languages, try has optional finally clause. The statements given in finally block will be executed even after the exceptions are handled.

```
try:
    f = open("aa.txt",'r')
    f.write('exception handling')
except:
    print('file write exception')
finally:
    f.close()
    print('normal flow')
'''
sample output:
file write exception
normal flow
'''
```

Raising Exceptions

Exception can be raised from a function:

raise ExceptionClass('Something Wrong')

Example:

```
ex=RunTimeError('Something Wrong')

raise ex

OR

Raise RunTimeError('Something Wrong')
```

```
try:
    a = int(input("Enter a positive integer: "))
    if a <= 0:
        raise ValueError("That is not a positive number!")
except ValueError as er:
    print(er)

'''Sample output:
Enter a positive integer: -7
That is not a positive number!
'''
```

Custom Exception/User Defined Exception

In Python custom exception or otherwise called as user defined exception can be handled by creating a new user defined class which is a derived class from Exception class.

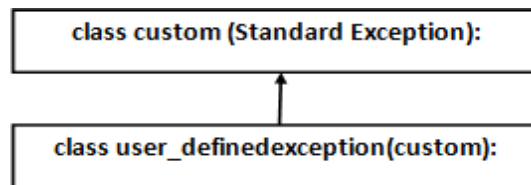


Fig. 2.2: Inheriting the Standard Exception class

In the following example two user defined exception classes are derived from the parent class Error which inherits the standard Exception class. The number guessed is 10. When any number greater than 10 is given as input TooLargeErr exception is thrown and when the number is less than 10, TooSmallErr exception is thrown.

```
class Error(Exception):
    pass
class TooSmallErr(Error):
    pass
class TooLargeErr(Error):
    pass

n=10
while True:
    try:
        x=int(input('enter a number'))
        if x<n:
            raise TooSmallErr
        elif x>n:
            raise TooLargeErr
        break
    except TooSmallErr:
        print('value is small, try again!..')
        print()
    except TooLargeErr:
        print('value is large, try again!..')
        print()
print('Wow! Guess is correct!')

'''output
enter a number23
value is large, try again!..

enter a number1
value is small, try again!..

enter a number10
Wow! Guess is correct!'''
```

REGULAREXPRESSIONS

Regular Expressions can also be called as RE/regex/regex patterns .RE's are specialized programming languages embedded inside Python. RE's are available by importing **re** module. RE patterns are compiled into a series of bytecodes when executed by a matching engine written in C language. REs could not perform al

S11BLH21- Programming in python

string processing tasks. REs are applicable in Pattern recognition problems. RE module has to imported for calling re methods like: split(), findall(), search() etc.

Syntax:

```
import re
```

RE matching characters

Character matching is very important for identifying patterns and matching them with the given input. The following table describes some of the important matching characters used in Python REs.

Table: 2.1 Python Character Matching

Matching Character	Description
[]	Finding a range of characters [a-z]
\w	Alphanumeric character [a-zA-Z0-9]
\W	Non alpha numeric characters :^ [a-zA-Z0-9]
*	Repeating a character [0] or more times
()	Grouping or including
+	1 or more
?	0 or 1

S11BLH21- Programming in python

{x}	Exact number of matches
{a,b}	In range from a to b
\any_number	Matching the group of same number.
\A	Only at the start of the string.
\Z	Only at the end of the string
\b	Empty string only at the beginning or end of a word.
\B	Empty string match not at the beginning or end of a word
\d	[0-9]
\D	^[0-9]
\s	Space
\S	Non space

RE Methods

The search() method

Method:search()

Description: Returns true if the search string is found.

Example:

```
import re
m = re.search('info','information')
if m:
    print(m,"is found")
else:
    print('not found')

''' output
<re.Match object; span=(0, 4), match='info'> is found
'''
```

S11BLH21- Programming in python

The above code returns the Match object with a span position from 0 to n-1 when the search information is found.

The split() method

Method:split()

Description: For creating space in the string.

Example:

```
import re
print(re.split(r'(\s)', 'This is a string'))
print()
print(re.split(r'[a-i]', 'This is a string'))
'''
output:
['This', ' ', 'is', ' ', 'a', ' ', 'string']

['T', '', 's ', 's ', ' str', 'n', '']
'''
```

In the above code, split() method is applied twice on the string, 'This is a string'. When the matching character \s is applied, the spaces in the string are split up. When the regular expression r'([a-i])' is applied, the string is split ignoring the range of characters from a to i.

The findall () method

Method:findall()

Description: Finds all the matches and returns them as a list of strings.

S11BLH21- Programming in python

Example:

```
import re
n='123 1234 12345 636525 1478523'
print(re.findall('\d{5,7}',n))
'''output
returns digits of length from 5 to 7
['12345', '636525', '1478523']
'''
```

The match() method

Method:match()

Description:To match the RE pattern to string with optional flags.

Example:

```
import re
list=['csea','cseb','cse a and b']
for e in list:
    z=re.match('(c\w+)',e)
    if z:
        print(z.groups())
'''
Sample output:
The first word of the list items matching the letter c is grouped up
('csea',)
('cseb',)
('cse',)
'''
```

S11BLH21- Programming in python

The finditer() method Method:

finditer() **Description:**Generating

an iterator.**Example:**

```
import re
str='welcome to cse dept and it dept of Soc'
for i in re.finditer('dept',str):
    localtuple=i.span()
    print(localtuple)

'''output:
returns start index and end index of the string
'dept' which occurs in 2 places:

(15, 19)
(27, 31)
'''
```

The compile() method

Method:compile()

Description: Compiling a pattern without rewriting it.

S11BLH21- Programming in python

Example:

```
import re
pattern=re.compile('Python')
result=pattern.findall('Welcome to Python programming. Python is Object Oriented.')
print(result)
result2=pattern.findall('Learning Python is Simple')
print(result2)
'''output
['Python', 'Python']
['Python']
'''
```

In the above code the compiled pattern is ‘Python’. The result objects return each and every occurrence of the matched pattern line by line. Other Regular Expression methods are given in Table 2.2 and RE Compilation flags are given in Table 2.3.

Table 2.2 Other RE methods

Method/Attribute	Purpose
group()	Returns the string matched by the RE
start()	Returns the starting position of the match
end()	Returns the ending position of the match
span()	Returns a tuple containing the starting and ending positions of the match
sub()	Replaces the RE pattern and returns the modified string

Table 2.3. RE Compilation Flags

Flag	Syntax	Description
ASCII	re.A	Makes several escapes like \w,\b,\s and \d and match only on ASCII characters
DOTALL	re.S	Match any character including newline
IGNORECASE	re.I	Case insensitive matches
MULTILINE	re.M	Multiline matching affecting ^ and \$
LOCALE	re.L	Locale aware match(Localization API)
VERBOSE	re.X	Enables verbose RE

Example:

```
import re
list='''csea
nseb
dseal and b'''

m1=re.findall(r'^\w',list)
m2=re.findall(r'^\w',list,re.MULTILINE)

print(m1)
print()
print(m2)

'''output
['c'] <- returns only the first character of first line

['c', 'n', 'd'] <-returns all first characters since it is multiline
'''
```

Case Studies on Pattern Matching:

Case Study 1: Phone number verification

```
import re
ph='412-555-342-4533'
if re.search('\w{3}-\w{3}-\w{3}-\w{4}',ph):
    print('valid phone no')
else:
    print('invalid phone no')
'''
output:
valid phone no'''
```

Case Study 2: Validating First name & Last name

```
import re
name='arthi rathna'
if re.search('\w',name):
    print('valid full name')
else:
    print('invalid name')
'''
output:
valid full name'''
```

Case Study 3: Email Address Verification

```
import re
n='abc@gmail.com, x3@,abc.com,az2@abc.in'
print(re.findall('[\w._/]{1,20}@[ \w.-]{2,20}.[A-Za-z]{2,3}',n))
'''output
returns valid emailaddresses:
['abc@gmail.com', 'az2@abc.in']
'''
```

Case Study 4: Web Scrapping

```
import urllib.request
from re import findall
url='http://www.sathyabama.ac.in/sitepagethree.php?mainref=23/'
resp=urllib.request.urlopen(url)
html=resp.read()
htmlstr=html.decode()
pdata=findall('\d{4}\s-\s\d{3}\s-\s\d{4}',htmlstr)
for item in pdata:
    print(item)

'''
output:
1800 - 425 - 1770
'''
```

PYTHON MODULES

2.5.1.Definition

A module is a library of functions used to provide any service. To incorporate the service provided by any module, 'import' statement should be used in Python. Modules can be built in or user defined. Modules can be imported in the current program using

S11BLH21- Programming in python

the import statement.

Syntax:

```
import module_name
```

Example: Time module , Math module

Sample Programs on Built in modules

The time module

```
import time
ct_time=time.time()
print(ct_time)

'''Output:
    1559160028.4655905
'''
```

The math module

```
import math
print(math.sqrt(9))

'''
output:
3.0
'''
```

S11BLH21- Programming in python

Building Custom modules by Modularising functions

Files, containing the Python definitions and statements, can be created by the user, and the same file can be imported on another Python program using import statement. The following example explains importing a python module(File1) over another python code(File 2).

Example:

Let us have two different files File1 & File 2. If we want to import any module of File1 into File2 , then we need to import File1 module in File2 using ‘import’ statement.

File1.py

```
def max(n1,n2):  
    if n1>n2:  
        result=n1  
    else:  
        result=n2  
    return result
```

File 2.py

```
import File1  
x,y=eval(input('enter x and y'))  
z=max(x,y)  
print("the max is",z)
```

On running File2.py, we get the maximum of two values as output.

INTRODUCTION TO PIP

In order to manage and install software packages Python uses PIP as Package Management System. PIP is written in Python and available in PyPI(Python Package Index). PIP is otherwise known as PiP Installs Python or PIP installs Packages.

Installing Packages via PIP

2.6.1.1 Steps for installing PIP

Step 1: Download get-pip.py and save this folder in the system's local drive to a folder on your computer.

Step 2: Open the command prompt and explore the folder containing get-pip.py.

Step 3: Run the command: python get-pip.py.

2.6.1.2. Using online python compiler

Python codes can also be executed online without installing Python IDLE or PIP packages. One of the weblink used for running python codes online is: 'https://www.onlinegdb.com/online_python_compiler#'.

2.7. USING PYTHON PACKAGES FOR ADVANCED PROGRAMMING

2.7.1. Python editors for Advanced Python Programming

The following are some of the python editors where Python libraries necessary for advanced scientific programming are almost readily available. If the Python library is not available then the command '`pip install lib_name`', could be given for installing the specific library.

S11BLH21- Programming in python

- Jupyter Notebook
- Pycharm Community Edition & Professional Edition
- Wing IDE
- NINJA IDE
- Spyder
- Pyzo

Python Libraries for running real time projects

Numpy

Numpy is a package supporting multidimensional arrays and it is designed for scientific computation purpose. Simple code to create a 3×5 array using numpy is given as follows:

```
import numpy as np

a =

np.arange(15).reshape(3,

5)print(a)

print ('type of a', type(a))
```

#Output:

Table 2.4: Universal Functions in Numpy

Function name	Purpose	Example
np.array()	For creating arrays	a = np.array([0, 1, 2, 3])
np.arange()	For formatting the array. Start index, end index, step which are the optional attributes.	b = np.arange(1, 9, 2) output: [1,3,5,7]
np.linspace()	For array line spacing with attributes start, end and num-points.	c = np.linspace(0, 1, 6)
np.reshape()	To specify the array dimensions	np.reshape(3,5) : forms a 3*5 array

Table 2.4: Universal Functions in Numpy(Contd...)

Functionname	Purpose	Example
np.sqrt()	Finding square root of an array	d=np.array([[100, 144, 256],[144, 4, 81]]) print(np.sqrt(d)) Output: [[10. 12. 16.] [12. 2. 9.]]
np.exp()	Finding exponential power	np.exp(2)
np.add()	Adding values to an array	np.add(a,10) [[10 11 12 13 14] [15 16 17 18 19] [20 21 22 23 24]]

S11BLH21- Programming in python

Scipy

Scipy library is used for performing mathematical and scientific calculations. Scipy can also be used for Engineering applications.

Syntax:

from scipy import module_name

Example:

```
import scipy
from scipy.constants import pi
print("sciPy - pi = %.16f"%scipy.constants.pi)
```

Output:

```
sciPy - pi = 3.1415926535897931
```

The following are the real time applications which can be implemented using Scipy:

- Signal Processing
- Image manipulation
- Interpolation
- Optimization and fit
- Statistics and random numbers

- File input/output
- Special Function
- Linear Algebra Operation
- Numerical Integration
- Fast Fourier transforms

Matplotlib

Matplotlib library is used for plotting graphs. The basic methods in matplotlib are:

- Plot()- To plot X, Y axes.
- Show()- To display the plotted graph.

Example:

```
%matplotlib inline  
  
importmatplotlib.pyplot as myplt  
  
myplt.plot([1,2,3,4])  
  
myplt.ylabel('numbers')  
  
myplt.show()
```

Output

